ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 30

Base de datos: migraciones, seeders y factories

- La configuración de la base de datos
- Qué es una migración
- Las tablas de la academia
- Ejecutar y deshacer migraciones
- Modificar una tabla
- Factories: datos falsos con Faker
- Seeders: rellenar la base de datos
- El Query Builder
- La academia, desde la base de datos



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Hasta ahora, cada vez que necesitábamos una tabla, abríamos HeidiSQL o un fichero `base-de-datos.sql` y escribíamos `CREATE TABLE` a mano. Funciona, pero tiene un problema: si un compañero añade una columna, ¿cómo os enteráis vosotros? ¿Y el servidor? ¿Os pasáis el SQL por WhatsApp? 😅

Laravel lo resuelve con las **migraciones**: la estructura de la base de datos escrita en PHP, guardada en Git y aplicada con un comando. Hoy además rellenaremos las tablas con datos de prueba en segundos (*seeders* y *factories*) y consultaremos con el **Query Builder**. Los cursos dejan de vivir en `config/academia.php` : por fin, base de datos de verdad.

•

👌 Cómo vamos a trabajar



1

📘 La configuración de la base de datos



2

🤔 Qué es una migración



3

👍 Las tablas de la academia



4

🚀 Ejecutar y deshacer migraciones



5

🔍 Modificar una tabla



6

😮 Factories: datos falsos con Faker



7

👌 Seeders: rellenar la base de datos



8

📘 El Query Builder



9

🚀 La academia, desde la base de datos



10

🚀 Ejercicios



11

👋 Lo que tenéis que llevaros de hoy





CLASE 30

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto y ejecutad:

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
php artisan migrate:fresh --seed
```

Esquema

```
database/migrations/        ← NUEVAS: categorías, cursos, alumnos, matrículas e imagen del
curso
database/factories/         ← NUEVAS: CategoriaFactory, CursoFactory y AlumnoFactory
database/seeders/           ← NUEVOS: CategoriaSeeder, CursoSeeder y AlumnoSeeder;
DatabaseSeeder cambia
app/Models/                 ← NUEVOS: Categoria, Curso y Alumno (los veremos a fondo en la
clase 31)
config/academia.php         ← ya no tiene los cursos
app/Http/Controllers/       ← CursoController y PaginaController leen de la base de datos
routes/api.php y console.php ← también
resources/views/            ← la tarjeta y la ficha del curso usan objetos
```



⚠️ `migrate:fresh` **borra todas las tablas** y las vuelve a crear. En nuestro ordenador es cómodísimo; en un servidor con datos reales, **jamás**.



💡 Si hicisteis los ejercicios de las clases 28 y 29 (profesores o la tabla de precios), leían los datos de `config/academia.php` : adaptadlos a la base de datos o borradlos.



CLASE 30 · PUNTO 1

📘 La configuración de la base de datos

Ya lo dejamos preparado en la clase 27. Los datos de conexión van en el `.env` [1]:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=academia_laravel
DB_USERNAME=root
DB_PASSWORD=
```

Y `config/database.php` los lee con `env()`. Allí se ve que Laravel sabe hablar con MySQL, MariaDB, PostgreSQL, SQL Server y SQLite: cambiar de una a otra es cambiar `DB_CONNECTION`. Es la misma idea que el DSN de PDO de la clase 14, porque **por debajo Laravel usa PDO**, con consultas preparadas siempre.

Para comprobar que la conexión funciona:

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
php artisan db:show
```



Una **migración** es una clase de PHP con dos métodos [2]:

- `up()`: lo que hay que hacer (crear una tabla, añadir una columna...).
- `down()`: cómo deshacerlo.

Viven en `database/migrations/` y su nombre empieza por la **fecha y hora**, para que se ejecuten en orden. Laravel apunta en la tabla `migrations` cuáles ha ejecutado ya, así que cada una se aplica **una sola vez**.

Pensad en las migraciones como el **control de versiones de la base de datos**: igual que Git guarda la historia del código, las migraciones guardan la historia de las tablas.

Esquema

```
Sin migraciones                    Con migraciones
———————————————                    ———————————————
«Oye, que he añadido una columna,   git pull
 ejecuta este SQL…»                 php artisan migrate   ✅
```

Las crea Artisan:

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
1 php artisan make:migration create_categorias_table
```

Si el nombre sigue el patrón `create_XXX_table`, Laravel ya nos da preparado el `Schema::create('XXX', ...)`.



Este es el modelo de datos que vamos a usar hasta el final del curso:

Esquema

```
1  categorias 1 ──── N cursos N ──── N alumnos
2                          │              │
3                          └ alumno_curso ┘   (tabla intermedia: las matrículas)
```

Laravel tiene convenciones de nombres, y si las seguimos, más adelante nos ahorrará mucho trabajo:

Qué	Convención	Ejemplo
Tabla	Plural, minúsculas y guion bajo	`cursos`, `alumnos`
Clave primaria	`id`	`id`
Clave foránea	Singular de la tabla + `_id`	`categoria_id`
Tabla intermedia	Los dos singulares **en orden alfabético** con `_`	`alumno_curso`
Fechas automáticas	`created_at` y `updated_at`	`$table->timestamps()`

La migración de los cursos (ver fichero):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function up(): void
{
    Schema::create('cursos', function (Blueprint $table) {
        $table->id();
        $table->foreignId('categoria_id')->constrained()->restrictOnDelete();
        $table->string('titulo', 120);
        $table->string('slug', 140)->unique();
        $table->text('descripcion')->nullable();
        $table->unsignedSmallInteger('horas');
        $table->decimal('precio', 8, 2);
        $table->unsignedSmallInteger('plazas')->default(20);
        $table->date('fecha_inicio')->nullable();
        $table->boolean('publicado')->default(true);
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('cursos');
}
```

Comparadlo con el `CREATE TABLE` de la clase 14: es lo mismo, pero en PHP y sin preocuparnos de si la base de datos es MySQL o PostgreSQL.

Método	Columna
`id()`	`BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY`
`string('titulo', 120)`	`VARCHAR(120)` (255 si no ponemos tamaño)
`text('descripcion')`	`TEXT`
`integer()`, `unsignedSmallInteger()` …	Enteros de distintos tamaños
`decimal('precio', 8, 2)`	`DECIMAL(8,2)`: para dinero, **nunca** `float`
`boolean('publicado')`	`TINYINT(1)`
`date()`, `dateTime()`, `timestamp()`	Fechas
`timestamps()`	`created_at` y `updated_at`, que Eloquent rellena solo
`foreignId('categoria_id')->constrained()`	`BIGINT UNSIGNED` + clave foránea a `categorias.id`

Y los **modificadores**, que se encadenan: `->nullable()`, `->default(20)`, `->unique()`, `->after('descripcion')`, `->comment('...')`.

Las claves foráneas

`foreignId('categoria_id')->constrained()` deduce del nombre que apunta a la tabla `categorias`. Después decidimos qué pasa si se borra la categoría:

- `->restrictOnDelete()`: no deja borrarla si tiene cursos (lo que queremos aquí).
- `->cascadeOnDelete()`: borra también sus cursos. Lo usamos en la tabla de matrículas: si se borra un alumno, desaparecen sus matrículas.
- `->nullOnDelete()`: deja la columna a `NULL` (tiene que ser `nullable()`).

La tabla intermedia

`alumno_curso` guarda las matrículas, y aprovechamos para guardar también la **nota**. El índice `unique` impide matricular dos veces al mismo alumno en el mismo curso:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
Schema::create('alumno_curso', function (Blueprint $table) {
    $table->id();
    $table->foreignId('alumno_id')->constrained()->cascadeOnDelete();
    $table->foreignId('curso_id')->constrained()->cascadeOnDelete();
    $table->decimal('nota', 4, 2)->nullable();
    $table->timestamps();

    $table->unique(['alumno_id', 'curso_id']);
});
```



Comando	Qué hace
`php artisan migrate`	Ejecuta las migraciones pendientes
`php artisan migrate:status`	Cuáles se han ejecutado y cuáles no
`php artisan migrate:rollback`	Deshace el **último lote** (llama a los `down()`)
`php artisan migrate:rollback --step=1`	Deshace solo la última migración
`php artisan migrate:refresh`	Deshace todas y las vuelve a ejecutar
`php artisan migrate:fresh`	**Borra todas las tablas** y ejecuta todo desde cero
`php artisan migrate:fresh --seed`	Lo mismo y, además, rellena con los *seeders*



⚠️ **La regla de oro:** una migración que ya se ha ejecutado en otro ordenador o en el servidor **no se modifica**. Si hay que cambiar algo, se crea una migración **nueva**. Es como un commit publicado en Git: no se reescribe.



CLASE 30 · PUNTO 5

🔍 Modificar una tabla

En la clase 32 subiremos una imagen para cada curso, así que necesitamos una columna nueva. Siguiendo la regla de oro, creamos otra migración:

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
php artisan make:migration add_imagen_to_cursos_table
```

Con el patrón `add_XXX_to_YYY_table`, Laravel nos da preparado un `Schema::table()` (ver fichero):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function up(): void
{
    Schema::table('cursos', function (Blueprint $table) {
        $table->string('imagen')->nullable()->after('descripcion');
    });
}

public function down(): void
{
    Schema::table('cursos', function (Blueprint $table) {
        $table->dropColumn('imagen');
    });
}
```

Otras operaciones habituales dentro de `Schema::table()`: `renameColumn('viejo', 'nuevo')`, `$table->string('titulo', 200)->change()` para cambiar el tipo, y `dropForeign(['categoria_id'])`.



CLASE 30 · PUNTO 6

😮 Factories: datos falsos con Faker

Para probar una web necesitamos datos: muchos y variados. Escribirlos a mano es un aburrimiento, así que Laravel trae las ***factories***: «fábricas» que saben crear registros falsos pero creíbles [3]. Por debajo usan **Faker**, una librería que inventa nombres, correos, fechas, teléfonos…

Las *factories* trabajan con **modelos**, así que hoy creamos tres modelos mínimos. Lo más cómodo es crear modelo, migración, *factory* y *seeder* de una vez:

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
1 php artisan make:model Curso -mfs
```

El modelo por ahora solo lleva `HasFactory` (en la clase 31 lo llenamos):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
1 class Curso extends Model
2 {
3     // Permite usar Curso::factory() en los seeders y en las pruebas
4     use HasFactory;
5 }
```

Y la *factory* define cómo es un curso cualquiera (`CursoFactory.php`):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function definition(): array
{
    // Faker no inventa textos en español, así que combinamos temas y niveles sin repetir
    $temas = ['Python', 'SQL', 'Photoshop', 'WordPress', 'Inglés', 'Marketing digital',
'React', 'Git', 'Linux', 'Docker'];
    $niveles = ['básico', 'intermedio', 'avanzado', 'práctico'];
    [$tema, $nivel] = fake()->unique()->randomElement(Arr::crossJoin($temas, $niveles));
    $titulo = "$tema $nivel";

    return [
        // Si no le decimos la categoría, crea una nueva
        'categoria_id' => Categoria::factory(),
        'titulo' => $titulo,
        'slug' => Str::slug($titulo),
        'descripcion' => "Aprende $tema con un enfoque $nivel, con ejercicios y un proyecto
final.",
        'horas' => fake()->randomElement([20, 25, 30, 40, 50, 60, 80]),
        'precio' => fake()->numberBetween(9, 45) * 10,
        'plazas' => fake()->numberBetween(10, 25),
        'fecha_inicio' => fake()->dateTimeBetween('now', '+6 months'),
        'publicado' => fake()->boolean(85),
    ];
}
```

Faker	Genera
`fake()->firstName()`, `lastName()`	Nombres y apellidos (españoles con `APP_FAKER_LOCALE=es_ES`)
`fake()->safeEmail()`	Un correo de `example.com`, que nunca existe
`fake()->numberBetween(10, 25)`	Un entero
`fake()->randomElement([...])`	Un elemento de la lista
`fake()->boolean(85)`	`true` el 85 % de las veces
`fake()->dateTimeBetween('now', '+6 months')`	Una fecha
`fake()->numerify('6## ### ###')`	Cambia cada `#` por un dígito
`fake()->unique()->...`	Sin repetir valores
`fake()->optional()->...`	A veces `null`

`Str::slug('Diseño web con Figma')` devuelve `diseno-web-con-figma`: el texto preparado para usarlo en una URL.

Y se usan así (probadlo en `php artisan tinker`):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
Curso::factory()->create();                          // uno, guardado en la base de datos
Curso::factory(10)->create();                        // diez
Curso::factory()->create(['precio' => 99]);          // con algún dato fijo
Curso::factory()->make();                            // uno, SIN guardar
Curso::factory()->sinPublicar()->create();           // con un "estado" definido en la factory
```



CLASE 30 · PUNTO 7

👌 Seeders: rellenar la base de datos

Un ***seeder*** («sembrador») es una clase que mete datos en la base de datos: los **fijos** que la aplicación necesita (las categorías) y los de **prueba** (los alumnos inventados) [4].

Las categorías son siempre las mismas, así que las insertamos tal cual (`CategoriaSeeder.php`):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function run(): void
{
    DB::table('categorias')->insert([
        ['nombre' => 'Programación', 'slug' => 'programacion', 'created_at' => now(), 'updated_at' => now()],
        ['nombre' => 'Diseño', 'slug' => 'diseno', 'created_at' => now(), 'updated_at' => now()],
        ['nombre' => 'Ofimática', 'slug' => 'ofimatica', 'created_at' => now(), 'updated_at' => now()],
    ]);
}
```

En `CursoSeeder` mezclamos los cinco cursos «de verdad» con diez inventados, y en `AlumnoSeeder` creamos 40 alumnos y matriculamos a cada uno en entre uno y tres cursos:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
Alumno::factory(40)->create();

$alumnos = DB::table('alumnos')->pluck('id');
$cursos = DB::table('cursos')->pluck('id');

foreach ($alumnos as $alumnoId) {
    foreach ($cursos->random(rand(1, 3)) as $cursoId) {
        DB::table('alumno_curso')->insert([
            'alumno_id' => $alumnoId,
            'curso_id' => $cursoId,
            'nota' => fake()->optional(0.6)->randomFloat(2, 3, 10),
            'created_at' => now(),
            'updated_at' => now(),
        ]);
    }
}
```

El que manda es `DatabaseSeeder`, que llama a los demás **en orden**:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function run(): void
{
    User::factory()->create([
        'name' => 'Ramón Abramo',
        'email' => 'ramon@academia-alpe.test',
    ]);

    // El orden importa: los cursos necesitan categorías, y las matrículas, cursos
    $this->call([
        CategoriaSeeder::class,
        CursoSeeder::class,
        AlumnoSeeder::class,
    ]);
}
```

terminal — php-paso-a-paso/clase30 — Visual Studio Code

terminal

```
php artisan db:seed                          # ejecuta DatabaseSeeder
php artisan db:seed --class=CursoSeeder      # solo uno
php artisan migrate:fresh --seed             # desde cero, con datos: el que más usaréis
```



💡 El usuario que crea `UserFactory` tiene como contraseña `password`. Lo necesitaremos en la clase 33.



CLASE 30 · PUNTO 8

📘 El Query Builder

El **Query Builder** («constructor de consultas») permite escribir consultas SQL **encadenando métodos de PHP**. Se usa con la fachada `DB` y, por debajo, genera SQL con **consultas preparadas**: adiós a la inyección SQL sin pensar en ello [5].

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
use Illuminate\Support\Facades\DB;

DB::table('cursos')->get();                          // SELECT * FROM cursos
DB::table('cursos')->find(3);                        // ... WHERE id = 3 (un objeto o null)
DB::table('cursos')->where('horas', '>', 40)->get();
DB::table('cursos')->where('titulo', 'like', '%PHP%')->first();
DB::table('cursos')->orderBy('precio', 'desc')->limit(3)->get();
DB::table('cursos')->pluck('titulo');                // solo una columna
DB::table('cursos')->where('publicado', false)->exists();
```

Query Builder	SQL
`->select('id', 'titulo')`	`SELECT id, titulo`
`->where('precio', '<', 200)`	`WHERE precio < 200`
`->where(...)->where(...)`	`... AND ...`
`->orWhere(...)`	`... OR ...`
`->whereIn('id', [1, 2, 3])`	`WHERE id IN (1, 2, 3)`
`->whereNull('fecha_inicio')`	`WHERE fecha_inicio IS NULL`
`->join('categorias', 'categorias.id', '=', 'cursos.categoria_id')`	`INNER JOIN ...`
`->leftJoin(...)`	`LEFT JOIN ...`
`->orderBy('titulo')` / `->orderByDesc('precio')`	`ORDER BY ...`
`->groupBy('categoria_id')`	`GROUP BY ...`
`->limit(10)->offset(20)`	`LIMIT 10 OFFSET 20`

Funciones de agregado

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
DB::table('cursos')->count();
DB::table('cursos')->avg('precio');
DB::table('cursos')->max('precio');
DB::table('cursos')->sum('horas');
DB::table('cursos')->orderByDesc('precio')->value('titulo');   // el título del más caro
```

Insertar, modificar y borrar

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
DB::table('categorias')->insert(['nombre' => 'Idiomas', 'slug' => 'idiomas']);
$id = DB::table('categorias')->insertGetId([...]);

DB::table('cursos')->where('id', 3)->update(['precio' => 350]);
DB::table('cursos')->where('id', 3)->increment('plazas', 5);

DB::table('cursos')->where('publicado', false)->delete();
```



⚠️ Un `update()` o un `delete()` **sin** `where()` afecta a **toda la tabla**. Igual que en SQL. Mirad dos veces antes de ejecutarlo.

¿Qué devuelve?

`get()` devuelve una **colección** (`Collection`): un array con superpoderes. Se recorre con `@foreach` como un array, pero además tiene decenas de métodos: `count()`, `isEmpty()`, `first()`, `pluck()`, `sum()`, `map()`, `filter()`, `random()`... Cada fila es un **objeto**, así que en las vistas se escribe `$curso->titulo` en lugar de `$curso['titulo']`.



🔎 ¿Queréis ver el SQL que genera? Cambiad `->get()` por `->toSql()` o por `->dd()`, que muestra la consulta y sus valores y detiene la ejecución.



CLASE 30 · PUNTO 9

🚀 La academia, desde la base de datos

Ahora sí: `CursoController` lee los cursos de MySQL, con el nombre de su categoría:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function index(Request $request): View
{
    $buscar = $request->query('buscar', '');

    $cursos = DB::table('cursos')
        ->join('categorias', 'categorias.id', '=', 'cursos.categoria_id')
        ->select('cursos.*', 'categorias.nombre as categoria')
        ->where('cursos.publicado', true)
        ->when($buscar !== '', function ($consulta) use ($buscar) {
            $consulta->where('cursos.titulo', 'like', "%{$buscar}%");
        })
        ->orderBy('cursos.titulo')
        ->get();

    return view('cursos.index', [
        'cursos' => $cursos,
        'buscar' => $buscar,
    ]);
}
```

`when()` añade una parte de la consulta **solo si** se cumple la condición. Es la forma elegante de los filtros opcionales, sin partir la consulta con `if`.

Y en las vistas cambiamos los arrays por objetos (`tarjeta-curso.blade.php`):

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<h3>{{ $curso->titulo }}</h3>
<span class="etiqueta">{{ $curso->categoria }}</span>
```

También pasan a la base de datos la portada, la API (`/api/cursos`) y el comando `academia:cursos`. Y estrenamos otro comando:

Esquema

```
$ php artisan academia:estadisticas
📊 Estadísticas de la academia
Cursos: 15
Alumnos: 40
Matrículas: 78
Precio medio: 241,33 €
Curso más caro: Python intermedio
```

¿Es cómodo el Query Builder? Sí. ¿Es lo más cómodo? Esperad a la clase 31: con **Eloquent**, `$curso->categoria->nombre` o `$curso->alumnos` funcionarán sin escribir ni un `join`. 😉



Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · La tabla de profesores.** Cread la migración, el modelo, la *factory* y el *seeder* de `profesores` (nombre, apellidos, email único, especialidad y años de experiencia). Cuidado: Laravel buscará la tabla `profesors`, así que indicad `protected $table = 'profesores';` en el modelo. El *seeder* debe crear un profesor fijo (vosotros) y cinco inventados. → `migración`, `Profesor.php`, `ProfesorFactory.php`, `ProfesorSeeder.php` y `DatabaseSeeder.php`
- ☐ **02 · Página de estadísticas.** Cread la página `/estadisticas` con una tabla por categoría (número de cursos, horas totales y precio medio, con `leftJoin`, `groupBy` y `DB::raw()`) y la lista de los cinco cursos con más alumnos. → `PaginaController.php`, `estadisticas.blade.php` y `routes/web.php`
- ☐ **03 · Matricular desde la consola.** Cread el comando `php artisan academia:matricular {alumno} {curso}`, que compruebe que existen los dos, que el alumno no esté ya matriculado y que queden plazas, y que inserte la matrícula. → `routes/console.php`



🏆 **Reto:** cread una migración que añada a `cursos` la columna `profesor_id` (opcional, con clave foránea a `profesores` y `nullOnDelete()`), y modificad `CursoSeeder` para asignar un profesor al azar a cada curso. Después, mostrad el nombre del profesor en la ficha del curso con un `leftJoin`.



CLASE 30 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Migración	La estructura de una tabla en PHP, con `up()` y `down()`, guardada en Git
Convenciones	Tablas en plural, `id`, claves `xxx_id`, tabla intermedia `a_b` en orden alfabético
`foreignId()->constrained()`	Clave foránea en una línea
`migrate` / `rollback` / `fresh --seed`	Aplicar / deshacer / empezar de cero con datos
Regla de oro	Una migración ya compartida no se toca: se crea otra
Factory	Fábrica de registros falsos con Faker
Seeder	Mete datos fijos y de prueba; `DatabaseSeeder` los llama en orden
Query Builder	SQL encadenando métodos, siempre con consultas preparadas
`when()`	Filtros opcionales sin `if`
`Collection`	Lo que devuelve `get()`: un array con superpoderes

📚 Fuentes y documentación oficial

1. Laravel: Database, Getting Started
2. Laravel: Migrations
3. Laravel: Eloquent Factories
4. Laravel: Seeding
5. Laravel: Query Builder
6. Laravel: Collections